

Analyse Santana

Note Globale : 72/100

Bad Technology Research — 20 juin 2026

1. Architecture & Structure : 80/100

80/100

Forces

- Organisation claire en 6 modules (core/, agent/, atlas_engine/, tools/, memory/, tg_handlers/)
- core/react_loop.py comme colonne vertébrale ReAct — point d'entrée unique
- tools/registry.py avec décorateur @tool() pour enregistrement automatique
- tools/vm_security.py — allowlist propre, défense 4 couches (binaire, flags, chemins, env)
- core/db.py — connexions SQLite thread-local avec WAL, busytimeout

Faiblesses

- tools/tools.py (624 lignes) est un god module — trop de responsabilités
- atlas_engine/atlas.py (27 KB) dense, fusion Steward+Writer mais doublons logiques
- agent/orchestrator.py et agent/orchestration.py — noms trop proches, rôles flous
- sys.path.insert dans classifier.py — chemins d'import non standardisés
- metrics.py inexistant — import cassé silencieusement

2. Cohérence & Complétude : 65/100

65/100

Forces

- Roadmap F1-F14 avec progression logique
- Migration JSON→SQLite documentée dans les docstrings
- code_exec.py avec sandboxing via unshare (mount+net namespace)

Faiblesses

- Code mort : patterns.py sections supprimées mais encore référencées
- vm_exec et vm_exec_script désactivés mais toujours dans tools.json — le LLM peut les appeler et recevoir une erreur
- metrics.py inexistant mais importé — décorateur @track() ne fait rien
- mcp_call retourne 'Erreur: outils MCP non disponibles'
- eval_history.json pollué : 50+ entrées de test score=0.5 (nettoyé le 20 juin)

3. Système de Mémoire : 78/100

78/100

Forces

- 3 couches : Flux (trace temporelle), Registres (personnes/dates), Livres (psychologie, famille, projets, vision_btr)

- Recherche vectorielle all-MiniLM-L6-v2 avec chunks par sections
- Cache LRU 40 entrées pour requêtes vectorielles
- Dédoublonnage cosinus (seuil 0.85)
- memory_injector.py — mémoire vivante compressée à 12 KB
- Détection de conflit avec similarité cosinus + marqueurs lexicaux
- Rotation automatique (max 500 souvenirs)

Faiblesses

- Deux bases SQLite (memory.db + metrics.db) — division arbitraire
- atlas.db._mort traîne dans memory/ — vestige
- MiniLM chargé 3 fois potentiellement — pas de singleton global
- Mémoire purement locale — pas de réplication, backup, ou sync GitHub

4. Agentique & Orchestration : 70/100

70/100

Forces

- ReAct loop complet : user → session → mémoire → outil → réflexion → réponse → Atlas learn → évaluation
- Détection de routines utilisateur (regroupement heure+sujet, seuil 3 occurrences)
- Évaluation passive des réponses (5 critères)
- Cost governor 3 niveaux (ALERT 80% / THROTTLE 95% / STOP 100%)
- Compression progressive du contexte (3 niveaux)
- agent/self.py — auto-description dynamique du système

Faiblesses

- Pas de vrai planificateur multi-étapes — agent/planner.py minimal (3.7 KB)
- orchestration.py et orchestrator.py — partage des responsabilités flou
- Proactivité déclenchée toutes les 30 min seulement
- Pas de queue de tâches, pas de priorisation, pas d'async pour tâches longues
- max_iter = 3 seulement (depuis le 20 juin) — faible pour tâches complexes

5. Tooling & Intégrations : 75/100

75/100

Forces

- 21 outils : web search, GitHub (4), code execution, memory, skills, tmux, rendu, social search
- GitHub ops via SSH key — pas de token API, pas de rate limit
- vm_security.py : allowlist, realpath(), détection expansion shell, sandboxing mount+net
- Registry pattern @tool() — extensible

Faiblesses

- Web search dépend uniquement de Serper.dev — pas de fallback
- web_navigate/web_screenshot pointent vers 127.0.0.1:5200 — services locaux fragiles
- Social search dépend de Xpoz — wrapper non documenté
- tools.json contient vm_exec/vm_exec_script qui sont des stubs — désynchronisation

6. Robustesse & Résilience : 68/100

68/100

Forces

- PID lock anti-double processus
- Crash flag avec mode dégradé automatique
- RotatingFileHandler (5×5 MB) + TokenFilter dans les logs
- Thread-safe SQLite WAL + busy_timeout
- Rate limiting par outil (30 appels/60s) + global (2s window)
- Timeouts sur tous les appels externes

Faiblesses

- Gestion d'erreur trop générique — except Exception sans escalade ni retry
- Pas de circuit breaker pour services externes
- 23 fichiers de tests (4099 lignes) mais rien en CI — jamais exécutés automatiquement
- Backup DB = SQLite seulement — pas de backup de l'arbre complet

7. Gestion de Contexte : 72/100

72/100

Forces

- Buffer session 20 messages + résumé auto toutes les 10 interactions
- Compression progressive : soft_warn 4K → soft_trim 8K → hard_trim 16K
- 5 derniers messages protégés de la compression
- Cache prompt système incrémental
- Désambiguïsation locale <5ms (pattern-matching)

Faiblesses

- Seuils en tokens estimés — pas de vraie mesure du contexte LLM
- CONTEXTUAL_TRIGGER_LENGTH = 120 caractères → trop de messages déclenchent planification inutile

8. Configuration & Flexibilité : 70/100

70/100

Forces

- .env chargé via load_env() avec override protection
- Soul/ sépare identité en fichiers modulaires (SOUL.md, RULES.md, STYLE.md, USER.md)
- CLAUDE.md documente les règles de développement

Faiblesses

- Aucun multi-modèle — DeepSeek V4 Flash en dur
- pyproject.toml version '1.0.0' jamais mise à jour
- Pas de feature flags
- Dépendances gonflées : weasyprint, yt-dlp, transformers lourd, edge-tts probablement inutilisés

9. Sécurité & Séparation : 62/100

62/100

Forces

- TokenFilter pour cacher les secrets dans les logs
- MCP serveur avec API key check
- vm_security.py : validate_command, validate_script, safe_env
- Rate limiting + audit des accès (securite.py)
- Souveraineté (F10) audite les appels externes en temps réel
- Traçabilité (F8) log structuré des actions

Faiblesses

- Commandes shell MCP : cat sur n'importe quel fichier = fuite potentielle
- Tests de sécurité volumineux (628 lignes) mais rien ne prouve qu'ils passent
- Pas de sandboxing Docker — vm_exec/vm_exec_script tournent sur l'OS
- Validation métacharactères contournable dans tools.py

10. Qualité Générale & Maintainabilité : 75/100

75/100

A horizontal progress bar with an orange segment on the left and a grey segment on the right. The orange segment contains the text '75/100'.

Forces

- Code bien commenté — docstrings partout, explications pour chaque fonction complexe
- Correctifs numérotés et documentés
- test_system_integrity.py bien pensé : imports sans exécuter les effets de bord
- CLAUDE.md maintenu — discipline de documentation

Faiblesses

- eval_history.json pollué (maintenant nettoyé)
- 30+ 'Correctif' dans le code = cicatrices qui auraient dû être refactorées
- Doublon Atlas/memory — 2 systèmes de mémoire sans hiérarchie claire
- Roadmap optimiste — multi-utilisateurs, cloud, revenus encore loin

Recommandations Prioritaires

1. Nettoyer eval_history.json — fait

50+ entrées de test score=0.5 supprimées le 20 juin 2026. Prochaine étape : migrer les métriques en SQLite comme les autres (metrics.db).

2. Réduire les dépendances

pyproject.toml a probablement 50% de packages jamais importés (weasyprint, yt-dlp, edge-tts...). Un pip install minimal allégerait le venv de 1-2 Go.

3. Automatiser les tests

23 fichiers de tests (4099 lignes) mais rien en CI. Ajouter un cron : `cd ~/santana && venv_new/bin/python3 -m pytest tests/test_system_integrity.py -v`

4. Unifier Atlas et Memory

Deux modules de mémoire vectorielle qui se chevauchent. Fusion ou séparation claire avec interfaces définies.

5. Vrai sandboxing shell

vm_exec/vm_exec_script tournent en direct. Docker ou subprocess avec PIPE restreinte obligatoire.

6. MiniML singleton

Modèle chargé 3 fois potentiellement. Implémenter un lazy singleton global.

7. Synchroniser tools.json et tools.py

vm_exec et vm_exec_script sont des stubs dans tools.py mais définis dans tools.json — le LLM peut les appeler pour rien.

Verdict

Santana est un projet ambitieux avec une architecture saine (72/100) mais qui souffre de dette technique accumulée : un système de mémoire doublon (Atlas/memory), des métriques inexploitable (eval_history.json pollué, maintenant corrigé), des dépendances gonflées, et 23 fichiers de tests qui ne sont jamais lancés automatiquement. Le code est bien commenté et la séparation en couches tient la route — mais les 'Correctifs' accumulés montrent que certaines fondations méritent une vraie refonte plutôt que des rustines.

Priorité : CI automatisée, nettoyage des dépendances, unification mémoire, et sandboxing shell.